K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

E-COMMERCE SALES ANALYTICS DASHBOARD

A Project Report Submitted in

partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Submitted By

S. No.	University ID	Name
1	2420030763	Sri Ram Krishnaji
2	2420030569	P. Venkat Ranjeeth
3	2420030372	Dinesh Kumar

Name of the Guide	Bonagani Prathusha

Under the Esteemed Guidance of

Bonagani Prathusha

Department of Computer Science and Engineering

AY 2026–2027

Semester: Odd

DECLARATION

We hereby declare that the project report entitled “E-Commerce Sales Analytics Dashboard” is a record of bona fide work carried out by Sri Ram Krishnaji (2420030763), P. Venkat Ranjeeth (2420030569), and Dinesh Kumar (2420030372), in partial fulfillment of the requirements for the award of the Bachelor of Technology degree in Computer Science and Engineering at K L (Deemed to be) University.

We further declare that the work presented in this report is based on our project study and implementation. Any external material, datasets, tools, and references used have been acknowledged appropriately. The results and screenshots included in the final submission should correspond to the actual application execution.

Sri Ram Krishnaji – 2420030763

P. Venkat Ranjeeth – 2420030569

Dinesh Kumar – 2420030372

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project report entitled “E-Commerce Sales Analytics Dashboard” is a bona fide work carried out and submitted by Sri Ram Krishnaji – 2420030763, P. Venkat Ranjeeth – 2420030569, and Dinesh Kumar – 2420030372, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering at K L (Deemed to be) University, during the academic year 2026–2027.

The work has been completed under the guidance and supervision of the undersigned.

Signature of the Guide	Course Coordinator	Head of the Department
Bonagani Prathusha Department of CSE	_____ Name / Designation	_____ Dr. P. Venkateswara Rao Department of CSE

ACKNOWLEDGEMENT

We express our sincere gratitude to everyone who supported and guided us throughout the development of the project titled “E-Commerce Sales Analytics Dashboard.” Their encouragement and valuable suggestions helped us complete the project work.

We are especially thankful to our project guide, Bonagani Prathusha, for the guidance, feedback, and encouragement provided during the planning, development, testing, and documentation of this project.

We express our gratitude to the Head of the Department of Computer Science and Engineering, Dr. P. Venkateswara Rao, and the faculty members of the department for providing the academic environment and facilities required to carry out this work.

We also thank our friends, classmates, and family members for their cooperation and motivation. Finally, we acknowledge the developers and maintainers of the software libraries and data resources used in this project.

Sri Ram Krishnaji – 2420030763

P. Venkat Ranjeeth – 2420030569

Dinesh Kumar – 2420030372

ABSTRACT

E-commerce platforms generate large volumes of transactional data, including orders, products, customers, quantities, prices, discounts, and delivery information. Analysing this data manually is time-consuming and makes it difficult for businesses to identify sales trends, customer preferences, and product performance. The project “E-Commerce Sales Analytics Dashboard” aims to transform raw sales records into meaningful visual insights through an interactive analytics dashboard.

The proposed system imports e-commerce sales data from a structured file such as CSV or Excel, validates and cleans the records, and derives useful measures such as total sales, order count, quantity sold, average order value, and category-wise performance. Interactive charts and filters allow users to explore sales by date, product category, region, customer segment, and payment or order status, depending on the fields available in the dataset.

The dashboard is intended to support business users, students, and analysts in understanding sales performance and making data-informed decisions. Python-based data processing can be implemented using Pandas and NumPy, while visualizations can be developed using Plotly or Matplotlib and presented through Streamlit. The exact metrics and conclusions shown in the final dashboard must be calculated from the dataset used during execution.

Keywords: E-Commerce, Sales Analytics, Dashboard, Data Visualization, Python, Pandas, Streamlit, Business Intelligence, KPI, Data Cleaning.

INDEX

S. No.	Chapters	Topics
	Acknowledgement	
	Abstract	
1	Introduction	1.1 Background; 1.2 Problem Statement; 1.3 Scope; 1.4 Objectives; 1.5 Importance; 1.6 Target Users
2	System Requirements	2.1 Hardware; 2.2 Software; 2.3 Development Tools
3	Technology Stack	3.1 Python; 3.2 Pandas and NumPy; 3.3 Streamlit; 3.4 Visualization; 3.5 Storage
4	System Architecture	4.1 High-Level Architecture; 4.2 Modules; 4.3 Execution Architecture
5	Design	5.1 User Interface; 5.2 Workflow / Data Flow Diagram
6	Implementation	6.1 Modules; 6.2 Data Import; 6.3 Data Cleaning; 6.4 KPI Calculation; 6.5 Visualization; 6.6 Filtering
7	Features	7.1 Main Features; 7.2 Feature Workflow; 7.3 Interactive Filters; 7.4 Dashboard
8	Testing	8.1 Functional; 8.2 Integration; 8.3 Tools; 8.4 Test Cases; 8.5 Evaluation
9	Deployment	9.1 Run Steps; 9.2 Environment; 9.3 Local Execution
10	Challenges and Limitations	10.1 Challenges; 10.2 Solutions; 10.3 Limitations
11	Future Enhancements	11.1 Planned Features; 11.2 Forecasting; 11.3 Cloud and Live Data
12	Conclusion	12.1 Summary; 12.2 Achievements; 12.3 Skills Learned
13	References	Datasets and Technical Documentation
14	Appendices	Screenshots; Code; Setup; User Manual; Team Contributions

CHAPTER 1 – INTRODUCTION

1.1 Background of the Project

E-commerce has become an important channel for buying and selling products. Every transaction produces data that can reveal how customers purchase, which products perform well, and how sales change over time. However, raw transaction files are difficult to interpret without suitable analysis and visualization.

A sales analytics dashboard organizes this information into key performance indicators (KPIs), summaries, and interactive charts. It helps users move from raw records to understandable patterns. This project develops a dashboard that processes e-commerce sales data and presents important business insights in one place.

1.2 Problem Statement

Businesses and student analysts may receive sales information in large CSV or Excel files. Manually sorting, calculating totals, comparing categories, and identifying trends can take significant time and may lead to errors. The problem addressed by this project is to build an easy-to-use system that cleans and analyses sales data and displays relevant information through interactive visualizations.

1.3 Scope of the Project

- Import sales data from CSV or Excel files.
- Validate column names, data types, missing values, and duplicate records.
- Calculate sales KPIs such as revenue, orders, quantity sold, and average order value when the required fields exist.
- Analyse sales by date, product, category, region, and customer segment where available.
- Provide interactive filters and charts for exploring the dataset.
- Display a data table and allow users to download filtered results.
- Support exploratory analysis; the dashboard does not automatically execute business transactions.

1.4 Objectives

1. Develop an interactive e-commerce sales analytics dashboard.
2. Convert raw sales records into a clean and analysis-ready dataset.
3. Calculate meaningful sales and order KPIs.
4. Visualize sales trends and category/product performance.
5. Allow users to filter data and compare different segments.
6. Provide a simple interface suitable for students and business analysts.

1.5 Importance of the Application

The dashboard reduces repetitive manual calculations and makes business information easier to understand. Visual summaries can help identify high-performing categories, periods of increased or decreased sales, and areas that may require further investigation. It also provides practical experience in data analytics, data cleaning, KPI design, and visualization.

1.6 Target Users / Audience

- Small and medium e-commerce business owners.
- Sales and marketing teams.
- Business analysts and data analysts.
- Students learning Python, data science, and business intelligence.
- Faculty members and project evaluators.

CHAPTER 2 – SYSTEM REQUIREMENTS

2.1 Hardware Requirements

Component	Minimum Requirement	Recommended
Processor	Intel Core i3 or equivalent	Intel Core i5 or equivalent
RAM	4 GB	8 GB or above
Storage	1 GB free space	2 GB or above
Display	Standard laptop/desktop display	1366 × 768 or higher
Input	Keyboard and mouse	Keyboard and mouse

2.2 Software Requirements

- Operating System: Windows 10/11, Linux, or macOS.
- Programming Language: Python 3.x.
- IDE: Visual Studio Code or Jupyter Notebook.
- Data Processing: Pandas and NumPy.
- Dashboard Framework: Streamlit.
- Visualization: Plotly and/or Matplotlib.
- File Support: CSV and Excel readers such as openpyxl, if Excel input is enabled.
- Version Control: Git (optional).

2.3 Development Tools and Frameworks

Python: Used for application logic, data processing, KPI calculations, and dashboard integration.

Pandas: Used to load, clean, transform, group, and summarize tabular sales data.

NumPy: Supports numerical operations and efficient calculations.

Streamlit: Creates the interactive web interface, file uploader, filters, KPI cards, and charts.

Plotly / Matplotlib: Creates interactive or static charts for trends and comparisons.

openpyxl: Can be used by Pandas to read Excel workbooks when Excel upload is supported.

CHAPTER 3 – TECHNOLOGY STACK

3.1 Application Logic – Python

Python is the primary programming language. It connects the input, data-cleaning, analysis, and visualization modules.

3.2 Data Processing – Pandas and NumPy

Pandas provides DataFrame operations for reading files, handling missing values, filtering rows, grouping records, and calculating summaries. NumPy supports numerical calculations.

3.3 Front-end – Streamlit

Streamlit provides a browser-based interface using Python. It supports file uploads, selection widgets, KPI displays, tables, and charts.

3.4 Data Visualization – Plotly and Matplotlib

Visualization libraries present sales trends, category comparisons, regional summaries, and other analytical results. Plotly can provide hover details and interactive charts.

3.5 Data Source – CSV / Excel

The application accepts structured transaction data. The available columns determine which analyses can be performed.

3.6 Export and Storage

Filtered records or summarized results may be exported as CSV. The original uploaded data is processed for the current application session unless persistent storage is separately configured.

CHAPTER 4 – SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

USER / ANALYST

↓ **Upload CSV / Excel file**

DATA INPUT AND VALIDATION

↓ Check columns, types, missing values and duplicates

DATA CLEANING AND TRANSFORMATION

↓ Prepare dates, quantities, prices and categories

ANALYTICS ENGINE

↓ KPIs • Grouping • Trend analysis • Segment comparison

INTERACTIVE STREAMLIT DASHBOARD

↓

KPI CARDS | CHARTS | FILTERS | DATA TABLE | EXPORT

4.2 Description of System Modules

Module	Description	Output
Data Input	Accepts CSV/Excel sales files and reads records.	Raw DataFrame
Validation	Checks required fields, data types, and file readability.	Validated data / warnings
Preprocessing	Handles missing values, duplicates, dates, and numeric columns.	Clean dataset
Analytics	Calculates KPIs and grouped summaries.	Metrics and summaries
Visualization	Displays charts, tables, and filter controls.	Interactive dashboard
Export	Allows filtered data or summaries to be downloaded.	CSV output

4.3 Application Execution Architecture

The application runs as a Python Streamlit process. The user opens the local application URL in a web browser, uploads a dataset, and interacts with the dashboard. Streamlit sends the selected inputs to Python functions, which process the data and return updated metrics and charts. A separate database server is not mandatory for the basic file-based implementation.

CHAPTER 5 – DESIGN

5.1 User Interface Design and Screenshots

The user interface is organized to make the analysis process clear. The final implementation may contain the following sections. Insert genuine screenshots captured from the running application in the spaces provided.

- Main dashboard: project title, file uploader, and overview.
- KPI summary: total revenue, order count, quantity sold, and average order value (where supported).
- Sales trend: revenue grouped by day, month, or year.
- Category and product analysis: comparisons of categories and top-selling products.
- Regional/customer analysis: breakdowns based on available location and customer fields.
- Filtered data: a table showing records after applying selected filters.

[Insert actual application screenshot here: Screenshot 1 – Main Dashboard]

[Insert actual application screenshot here: Screenshot 2 – KPI Summary and Sales Trend]

[Insert actual application screenshot here: Screenshot 3 – Category / Product Analysis]

[Insert actual application screenshot here: Screenshot 4 – Filters and Data Table]

5.2 System Workflow / Data Flow Diagram

Start → Upload Dataset → Validate File → Clean Data → Calculate KPIs

→ Apply Filters → Generate Charts and Tables → Display Insights → Export → End

The workflow begins when the user uploads a sales dataset. The system validates the file and prepares the data. It then calculates metrics and summaries, applies user-selected filters, and refreshes the visualizations. The user can inspect the results and optionally download the filtered records.

CHAPTER 6 – IMPLEMENTATION

6.1 Module-wise Implementation

Module	Input	Processing	Output
Input	CSV / Excel file	Read file and inspect columns	DataFrame
Cleaning	Raw records	Handle missing values, duplicates and formats	Clean DataFrame
Analytics	Clean DataFrame	Calculate KPIs and group summaries	Metrics
Dashboard	Metrics and summaries	Render filters, charts and tables	Visual report

6.2 Data Import

The data import module accepts a user-selected CSV or Excel file. Pandas can read a CSV file using read_csv() and an Excel file using read_excel(). The application should show a clear warning if the file is empty, unreadable, or does not contain usable columns.

6.3 Data Cleaning and Preprocessing

- Remove duplicate rows when duplicates are confirmed to be invalid.
- Convert date fields into a consistent datetime format.
- Convert price, sales, quantity, and discount fields into numeric types.
- Inspect missing values and choose an appropriate handling method.
- Standardize text labels such as category and region names.
- Validate negative or impossible values according to the dataset's business rules.
- Keep a record of excluded or corrected rows for transparency.

6.4 KPI Calculation

KPIs are calculated from the fields available in the dataset. Example definitions are:

KPI	Definition / Formula
Total Sales / Revenue	Sum of the transaction sales amount or quantity × unit price, depending on the dataset definition.
Order Count	Count of distinct order IDs, if an order identifier is available.
Quantity Sold	Sum of the quantity field.
Average Order Value (AOV)	Total revenue ÷ number of distinct orders.
Average Selling Price	Total revenue ÷ total quantity sold, when quantity is valid.
Growth Rate	((Current period value − Previous period value) ÷ Previous period value) × 100, when comparable periods exist.

The application must avoid calculating a KPI when the required source fields are unavailable. The dashboard should display a helpful message instead of presenting an invented value.

6.5 Visualization and Analysis

- Line chart: sales or revenue trend over time.
- Bar chart: sales by product category or region.
- Horizontal bar chart: top products by revenue or quantity.
- Pie/donut chart: category or payment-share analysis, if appropriate.
- Data table: filtered transaction records.

- Comparison chart: period-over-period or segment comparison when sufficient data exists.

6.6 Interactive Filtering

Users can select a date range, category, region, product, or other available field. Each filter updates the displayed KPIs, charts, and data table so that the user can focus on a particular segment.

CHAPTER 7 – FEATURES

7.1 List of Main Features

- CSV and Excel file upload.
- Dataset preview and validation messages.
- Automatic data cleaning and type conversion.
- Sales KPI cards.
- Time-based sales trend analysis.
- Category-wise and product-wise analysis.
- Region-wise or customer-wise analysis where data is available.
- Interactive filters and downloadable results.
- Responsive dashboard layout.

7.2 Working of Each Feature

File Upload: The user selects a sales file. The application reads it and previews the available records.

Data Validation: The system checks whether the file can be read and whether relevant columns are present.

KPI Summary: The analytics module calculates totals and averages from valid records.

Trend Analysis: Records are grouped by date period and displayed in a chart.

Category Analysis: Sales values are grouped by product category to compare performance.

Filters: Selected filter values are applied to the dataset and all dependent dashboard sections are refreshed.

Export: The user downloads the filtered data or summary in CSV format, if export is enabled.

7.3 Interactive Filters

Filters improve usability by allowing users to analyse only a selected time period, product group, region, or customer segment. The available filters depend on the columns in the uploaded file.

7.4 Dashboard Summary

The dashboard combines numerical KPIs with charts and tabular details. This arrangement provides both a quick overview and the ability to investigate the underlying records.

CHAPTER 8 – TESTING

8.1 Functional Testing

Functional testing checks whether the dashboard features behave according to their requirements. Testing should include valid and invalid uploads, filter changes, KPI calculations, chart rendering, and export operations.

8.2 Integration Testing

Integration testing verifies the flow from file upload through validation, preprocessing, analytics, visualization, and export. It confirms that a filter selection updates all related dashboard components consistently.

8.3 Tools Used for Testing

- Python runtime for executing the application.
- Streamlit for user-interface and interaction testing.
- Pandas and NumPy for validating data transformations and calculations.
- Plotly / Matplotlib for checking chart output.
- VS Code or Jupyter Notebook for debugging and test execution.

8.4 Test Cases and Results

Test ID	Test Scenario	Expected Result	Status
TC-01	Upload a valid CSV file	File is loaded and preview is displayed	To be verified
TC-02	Upload a valid Excel file	File is loaded when Excel support is configured	To be verified
TC-03	Upload an unsupported file	Clear validation message is displayed	To be verified
TC-04	Dataset contains missing values	Missing values are handled or reported	To be verified
TC-05	Apply a category filter	KPIs and charts reflect selected category	To be verified
TC-06	Select a date range	Only records in the selected period are analysed	To be verified
TC-07	Download filtered data	A valid CSV file is generated	To be verified
TC-08	Dataset lacks a KPI field	KPI is omitted or marked unavailable	To be verified

Note: Update the Status column to Pass or Fail only after executing each test against the final application.

8.5 Model / Analytics Evaluation

This project is a descriptive analytics dashboard rather than a predictive machine-learning model. Therefore, its primary evaluation focuses on data correctness, KPI calculation accuracy, filter consistency, chart correctness, usability, and response time. Validate KPI values against independent calculations on a small sample before final submission.

CHAPTER 9 – DEPLOYMENT

9.1 Steps to Run the Application

- Install Python 3.x.
- Open the project folder in Visual Studio Code.
- Create and activate a virtual environment.
- Install the packages listed in requirements.txt.
- Place the sample or project sales dataset in the project folder, or use the upload option.
- Run the Streamlit application.
- Open the local URL shown in the terminal.

Execution command:

```
python -m streamlit run app.py
```

9.2 Python Environment Configuration

A virtual environment isolates project dependencies from other Python applications. Example commands for Windows PowerShell are:

```
python -m venv venv
.\\venv\\Scripts\\Activate.ps1
pip install streamlit pandas numpy plotly openpyxl
```

9.3 Local Application Execution

After the application starts, Streamlit displays a local address, commonly http://localhost:8501. Open the displayed address in a browser, upload the dataset, and use the dashboard controls. The actual URL and port may differ depending on the local environment.

CHAPTER 10 – CHALLENGES AND LIMITATIONS

10.1 Issues Faced During Development

- Sales files may contain inconsistent column names and formats.
- Missing or duplicated records can affect totals and averages.
- Date values may use different formats or contain invalid entries.
- Large datasets may take longer to process and visualize.
- Some datasets may not contain fields required for every KPI.
- Different definitions of revenue, orders, and discounts can lead to inconsistent interpretation.

10.2 Solutions Applied

- Use file validation and clear error messages.
- Normalize column names and convert data types before analysis.
- Inspect missing values and document the chosen handling method.
- Use Pandas grouping and vectorized calculations for efficiency.
- Calculate only those KPIs supported by the available fields.
- Clearly label each KPI and document its formula.

10.3 Current Limitations

- Dashboard insights depend on the quality and completeness of the uploaded dataset.
- The application analyses historical data and does not guarantee future sales.
- Real-time integration with an e-commerce platform is not included unless separately implemented.
- Advanced customer segmentation and forecasting may require additional data and models.
- The basic implementation may run locally and may not include user authentication or role-based access.

CHAPTER 11 – FUTURE ENHANCEMENTS

11.1 Planned Features

- Connect directly to e-commerce databases and platform APIs.
- Add role-based access and secure user login.
- Schedule automatic data refresh and report generation.
- Provide richer drill-down analysis by product, customer, and location.
- Add PDF and Excel report exports.
- Deploy the dashboard to a cloud hosting platform.

11.2 Sales Forecasting

Time-series forecasting methods can be introduced to estimate future sales using historical trends. Forecasts should be evaluated with suitable error metrics and presented with clear assumptions and uncertainty ranges.

11.3 Cloud Deployment and Live Data

The dashboard can be deployed to a cloud platform so that authorized users can access it remotely. Future versions may connect to live order data through secure APIs and refresh the dashboard automatically.

CHAPTER 12 – CONCLUSION

12.1 Summary of the Project

The E-Commerce Sales Analytics Dashboard is designed to convert raw transaction records into useful business information. It provides a structured process for importing, validating, cleaning, analysing, and visualizing e-commerce sales data. KPI cards, interactive charts, filters, and data tables help users explore performance across time periods and business segments.

12.2 Achievements

- Designed the architecture of a file-based sales analytics dashboard.
- Identified data validation and preprocessing requirements.
- Defined key sales KPIs and their calculation methods.
- Designed interactive visualizations and filtering workflows.
- Prepared functional and integration test cases.
- Documented deployment steps, limitations, and future enhancements.

12.3 Skills Learned During Development

- Python programming and application development.
- Data cleaning and preprocessing using Pandas.
- Numerical analysis using NumPy.
- KPI design and business data interpretation.
- Dashboard development using Streamlit.
- Data visualization and chart selection.
- Software testing, debugging, documentation, and teamwork.

CHAPTER 13 – REFERENCES

7. McKinney, Wes. Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter. O'Reilly Media.
8. Pandas Documentation. https://pandas.pydata.org/docs/
9. NumPy Documentation. https://numpy.org/doc/
10. Streamlit Documentation. https://docs.streamlit.io/
11. Plotly Python Documentation. https://plotly.com/python/
12. Matplotlib Documentation. https://matplotlib.org/stable/
13. Python Documentation. https://docs.python.org/3/
14. Microsoft Excel Documentation. https://support.microsoft.com/excel
15. Any dataset used in the implementation: provide its exact title, publisher/source, URL, access date, and license here.

Dataset citation note: Include the actual dataset source used by the team. Do not cite a sample dataset unless it was genuinely used in the project.

CHAPTER 14 – APPENDICES

14.1 Application Screenshots

Attach screenshots captured from the final working application. Recommended screenshots include the home screen, uploaded dataset preview, KPI summary, sales trend, category analysis, filter selections, and export result.

14.2 Sample Code Snippets

```
import pandas as pd

# Load sales data
df = pd.read_csv("sales_data.csv")

# Basic cleaning
df.columns = df.columns.str.strip().str.lower()
df = df.drop_duplicates()

# Example KPI calculations (adjust names to match the dataset)
if "sales" in df.columns:
    total_sales = pd.to_numeric(df["sales"], errors="coerce").sum()
    print("Total Sales:", total_sales)

if "quantity" in df.columns:
    total_quantity = pd.to_numeric(df["quantity"], errors="coerce").sum()
    print("Quantity Sold:", total_quantity)
```

14.3 Installation and Setup Instructions

- Install Python 3.x.
- Open the project directory in VS Code.
- Create a virtual environment using python -m venv venv.
- Activate the environment.
- Install dependencies using pip install -r requirements.txt.
- Run python -m streamlit run app.py.
- Open the local URL displayed in the terminal.

14.4 User Manual

- Launch the dashboard in a browser.
- Upload the e-commerce sales CSV or Excel file.
- Review the dataset preview and validation messages.
- Select available filters such as date, category, or region.
- Review the KPI cards and visualizations.
- Inspect the filtered records in the data table.
- Download the filtered data if the export feature is available.

14.5 Team Contributions

Sri Ram Krishnaji – 2420030763: Project coordination, requirement analysis, dashboard planning, integration, documentation, and testing (revise this description to reflect the work actually completed).

P. Venkat Ranjeeth – 2420030569: Data collection and preparation, data cleaning, KPI calculation, and analytical validation (revise this description to reflect the work actually completed).

Dinesh Kumar – 2420030372: Visualization design, dashboard interface, testing, result interpretation, and documentation (revise this description to reflect the work actually completed).

All team members contributed to project discussions, integration, debugging, review, and final report preparation according to their actual work.